DOMAIN: DATA & BUSINESS INTELLIGENCE ANALYST

Customer Churn Analysis — Complete Technical Implementation Source Code

Executable Python Script / Jupyter Notebook Pipeline

In [1]: Modules & Data Ingestion

```
# Step 1: Library Ingestion and Environment Configuration
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler, LabelEncoder
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, roc_auc_score
import shap

# Load Telecom Dataset
df = pd.read_csv('telecom_churn.csv')
print(f"Dataset Shape: {df.shape}")
```

```
Dataset Shape: (7043, 21)
```

In [2]: Preprocessing & Data Hygiene

```
# Step 2: Data Scrubbing & Continuous Feature Coercion
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'].str.strip(), errors='coerce')
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)

# Drop Unique Key Column
df.drop(columns=['customerID'], inplace=True)

# Binary Encoding for Target Variable
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})
print(f"Target Churn Distribution:
{df['Churn'].value_counts(normalize=True)}")
```

```
Target Churn Distribution:
0    0.73463
1    0.26537
Name: Churn, dtype: float64
```

In [3]: Feature Engineering & Categorical Encoding

```
# Step 3: Synthesis of Business Metrics & One-Hot Encoding
df['AvgMonthlySpend'] = df['TotalCharges'] / (df['tenure'] + 1)
df['HighValueRisk'] = np.where((df['MonthlyCharges'] > 70) & (df['Contract'] == 'Month-to-month'), 1, 0)

cat_cols = df.select_dtypes(include=['object']).columns
df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)
print(f"Encoded Feature Set Count: {df_encoded.shape[1]}")
```

```
Encoded Feature Set Count: 32
```

In [4]: Stratified Splitting & Class Resampling

```
# Step 4: Partition Data and Handle Class Imbalance
X = df_encoded.drop(columns=['Churn'])
y = df_encoded['Churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)

# Apply SMOTE to Training Subset
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_res)
X_test_scaled = scaler.transform(X_test)
print(f"Balanced Training Set Samples: {X_train_res.shape[0]}")
```

```
Balanced Training Set Samples: 8254
```

In [5]: Model Training & Hyperparameter Tuning

```
# Step 5: XGBoost Classifier Optimization via GridSearch
xgb = XGBClassifier(eval_metric='logloss', random_state=42)
param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [3, 5],
    'learning_rate': [0.01, 0.1]
}

grid_search = GridSearchCV(xgb, param_grid, cv=5, scoring='roc_auc', n_jobs=-1)
grid_search.fit(X_train_scaled, y_train_res)

best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test_scaled)
y_proba = best_model.predict_proba(X_test_scaled)[:, 1]

print(f"Best Parameters: {grid_search.best_params_}")
print(f"ROC-AUC Score: {roc_auc_score(y_test, y_proba):.4f}")
```

```
Best Parameters: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200}
ROC-AUC Score: 0.8612
```

In [6]: Model Explainability & Export

```
# Step 6: SHAP Feature Importance Interpretation
explainer = shap.Explainer(best_model, X_train_scaled)
shap_values = explainer(X_test_scaled)

# Export Final Probability Scores for Dashboarding
results_df = pd.DataFrame({
    'Actual_Churn': y_test,
    'Predicted_Probability': y_proba,
    'Risk_Tier': np.where(y_proba > 0.7, 'High', np.where(y_proba > 0.3, 'Medium', 'Low'))
})
results_df.to_csv('churn_risk_predictions.csv', index=False)
print("Predictions and Risk Tiers successfully exported!")
```

```
Predictions and Risk Tiers successfully exported!
```